

Death Star Plans

Group 4 Fall 2025

TEAM INTRODUCTION

- Coy Baker - CS
- Brandie Ewing – ITC
- Trevor Kates - EE
- Justin Leach – ITC
- Berrick Rudakemwa - EE

OVERVIEW

- Common terms
- Problem statement
- Background
- Use case
- Impact
- Objective statement
- Requirements
- Assumptions
- Constraints
- Standards

- Hardware design trades
- Software design trades
- Design components
- Component trades
- Proposed design
- Budget costs
- Risk and mitigations
- Test plan
- Development and testing timeline
- Milestone and deliverables

COMMON TERMS

- IEEE: Institute of electrical and electronics engineers
- Encryption / Decryption - the process of transforming readable data (plaintext) into an unreadable format. Decryption is reverse.
- ESP-NOW – Peer to Peer wireless communication protocol for ESP32 devices(no Wi-Fi network)

PROBLEM STATEMENT

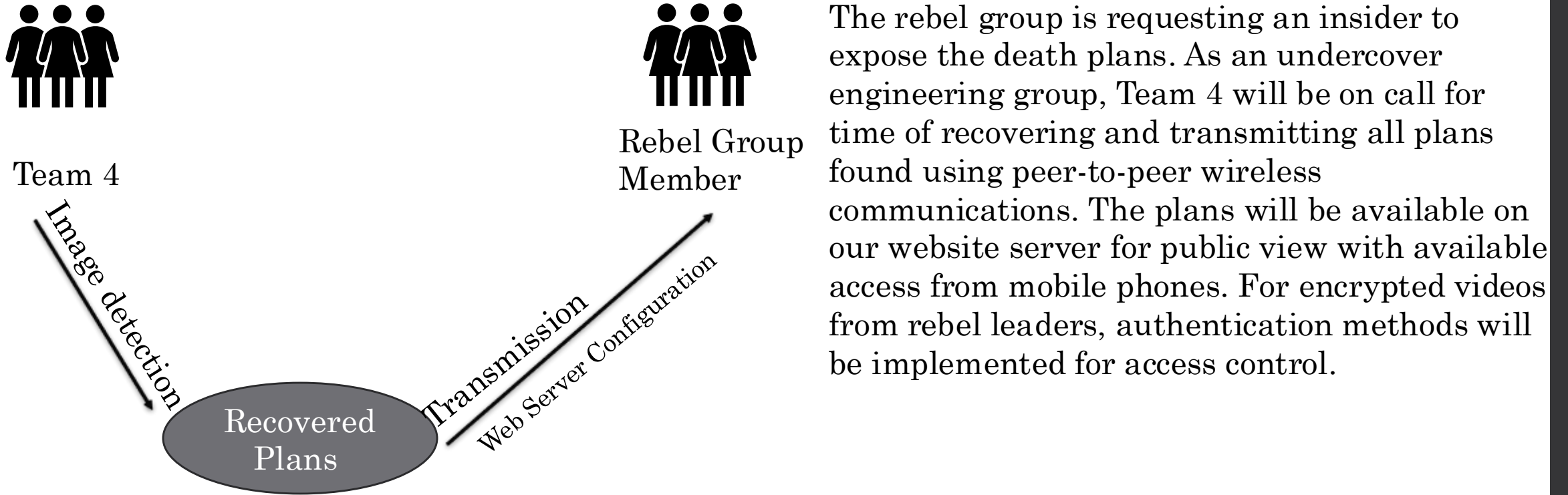
- In secure communication scenarios where wireless methods (Wi-Fi, & Bluetooth) are prohibited, the challenge is to transmit ten encrypted images from a Raspberry Pi to a laptop across a glass barrier within ten minutes. The solution must ensure confidentiality using only approved physical-layer methods. Team 4 is focusing on peer-to-peer communication solutions.

PROJECT BACKGROUND

- The Death Star Plans holds dangerous information that involves threatening the lives of innocent individuals. Therefore, developing a secure and covert method to transmit and recover the Death Star Plans is of utmost importance. Our proposed solution aligns with the technical review and addresses the core problem statement by establishing a reliable, encrypted, and wireless communication system between Team 4 and the Rebel Group to ensure mission success.

USE CASE

- Solutions successfully exposes Death Star Plans.



IMPACT

- Cultural: Protects community confidentiality but may risk misuse; mitigate with ethical content rules and leader communication.
- Economic: Low-cost, reliable parts; limited recyclability use backups and storage protocols to reduce costs.
- Environmental: Minimal impact; standard electronic footprint only.
- Global: Enables secure cross-border data transfer but risks misuse; mitigate with encryption and access control.
- Public Health: Protects medical data; misuse could expose private info secure systems prevent this.
- Public Safety: Reduces cyber risks but may overheat; use safety enclosures and system checks.
- Public Welfare: Builds trust through secure sharing; prevent misuse with ethical guidelines.
- Social: Enables private, offline transfers; restrict to authorized use to avoid abuse.

TECHNICAL REVIEW

- Hardware

- Raspberry Pi 4Model B
- SD Card
- Transmission Hardware
- Laptop
- Mobile Device

- Software

- Image Processing Software
- Transmission Software
- Encryption Software
- Web Hosting Software

OBJECTIVES

- Identify red circles of various sizes from 100 images.
- Encrypt images and has md5sum verification.
- Transfer images from the Raspberry Pi to the web server.
- Display the data on a web server to mobile devices.
- Authenticate a user on the web server to view an encrypted video message.

REQUIREMENTS

- The system shall compress the images to an average size of 50kb.
- The system shall upload 100 images and provide detection of red circles in each image.
- The system shall allow authenticated users to view transferred images through a web interface on mobile devices.
- The system shall compute the md5sum verification of images before transferring.
- The system shall transfer images via an ESP32 microcontroller radio signal.
- The system can display images with the red circle to the public in a web application.
- The system shall successfully transfer at least 10 images from the Raspberry Pi to the web server.
- The system shall have two factor authentication via a security question to display an encrypted video.
- The system shall support a minimum wireless data transmission rate of 10 kbps to ensure reliable transfer of encrypted image files from the raspberry pi through the esp32 to the laptop

ASSUMPTIONS

- The thumb drive will contain 100 images in one file.
- The transmitted signal will be able to go through the Plexiglass box.
- There will be minimal electromagnetic interference during operation.
- Images are delivered in a standard format.
- The quality of images will be approximately 50 KB.

Constraints

- The use of Wi-fi and Bluetooth are prohibited.

The system shall not cost more than \$300

The Raspberry Pi and ESP32 shall not accept user input or external commands while transmitting data.

The system shall not exceed 10 minutes (600 seconds) for time of transfer.

The system shall only process specific file formats (PNG).

The system shall be able to transfer data through the barrier.

ESP32 link shall use unlicensed ISM band settings within local legal limits.

Any hardware used shall not be physically attached to anything outside the operating environment

STANDARDS

- IEEE Std 1238.1-1994: IEEE Standard for Information Technology, OSI Application Program Interfaces—File Transfer, Access, and Management
- IEEE Std 828-2012 — Configuration Management in Systems and Software Engineering
- ANSI/NEMA WD 6-2021: Wiring Devices — Dimensional Specifications, National Electrical Manufacturers Association (NEMA), 2022
- IEEE Recommended Practice for Radio Frequency Safety Programs, 3 kHz to 300 GHz," in IEEE Std C95.7-2014.
- 29148-2011 - ISO/IEC/IEEE International Standard - Systems and software engineering -- Life cycle processes --Requirements engineering

HARDWARE DESIGN TRADES

	Design Options		
Criteria (Microcontroller Transfer Module)	ESP 32 S2	ESP 32 S3	ESP-32-WROOM-32
Req. 10	320 KB SRAM	512 KB SRAM + 8MB PSRAM	520 KB SRAM
Req. 90 (> 10kbps data rate)	5-20 kbps	200-800 kbps	100-500 kbps
Req. 70 (transfer rate < 600 s)	Single core	Dual-core @ 240 MHz (~200–800 kbps ESP-NOW v2)	Dual-core @ 240 MHz (~100–500 kbps)
Const. 10 (prohibited use of WIFI and Bluetooth)	Meets (no BT hardware, ESP-NOW PHY only)	Meets – BLE disabled; ESP-NOW uses Wi-Fi radio without network association	ESP-NOW Available but includes unused BT hardware
Const. 20 (cost)	\$10-15	\$10-15	\$10-15
Std. 20 (IEEE standard for IT, file transfer, access, management)	Meets basic standards for file transfer and AES	Meets basic standards for file transfer and AES	Meets basic standards for file transfer and AES

Evaluating the design trade-offs among the ESP32-S2, ESP32-S3, and ESP32-WROOM-32 modules, the **ESP32-S3** presents the most balanced choice in terms of performance, cost, and compliance with system constraints. While the ESP32-S2 offers simplicity and lower power consumption, its limited SRAM restricts data scalability. The ESP32-WROOM-32, though similar in processing capability, includes unnecessary Bluetooth hardware. The ESP32-S3 provides a **dual-core architecture**, **larger SRAM with external PSRAM support**, and significantly **higher data transfer rates** (200–800 kbps), all within the same cost bracket. Its ability to disable Bluetooth while maintaining ESP-NOW functionality through the Wi-Fi PHY layer meets the project's hardware restrictions, and its integrated AI acceleration provides headroom for future expansion. Thus, the ESP32-S3 represents an optimal trade-off.

SOFTWARE DESIGN TRADES 1

	Design Options		
Criteria (Image Detection Algorithm)	<i>OpenCV (Python /C++)</i>	<i>R-CNN</i>	<i>Single Shot Detector (SSD)</i>
<i>Req. 20 (adequate detection)</i>	Uses cv2.findContours() and cv2.matchTemplate() for detecting plans patterns	Uses “selective search” to propose regions with CNN	Uses CNN for predicting bounding boxes
<i>Req. 20 (raspberry pi capability)</i>	Runs on Pi 4's 8GB	Requires High GPU	Requires moderate GPU (better with TPU/edge support)
<i>Req. 60 (display)</i>	Uses cv2.imshow() for real-time display, overlay bounding boxes with cv2.rectangle() or keypoint visualization with cv2.drawKeypoints().	Requires external visualization tool (e.g., Matplotlib)	Supports real-time bounding box rendering but heavier computation
<i>Const. 50 (specific file formats)</i>	Compatible with .jpg, .png, and .bmp	Requires dataset preprocessing (.xml, .hdf5)	Works with standard image formats (.jpg, .png)
<i>Const. 20 (cost)</i>	Free (open source)	High (requires training resources)	Moderate (pretrained models available)
<i>Std. 20 (ISO systems and software engineering)</i>	Meets ISO/IEC 12207 (software process) and 25010 (quality) standards	Meets with additional integration	Meets with additional configuration

When comparing OpenCV, R-CNN, and SSD for image detection, **OpenCV** provides the best balance between performance and resource efficiency. R-CNN offers high detection accuracy but requires heavy GPU processing and extensive data preparation. SSD delivers faster detection with pretrained models but still needs moderate GPU. OpenCV, on the other hand, runs efficiently on the Raspberry Pi, supports standard image formats, and enables real-time visualization using built-in display functions. Overall, it achieves adequate detection accuracy while remaining cost-effective, lightweight, and compliant with ISO/IEC software engineering standards.

SOFTWARE DESIGN TRADES 2

	Design Options		
Criteria (Operating System)	Raspberry Pi OS	Ubuntu Linux	Kali Linux
Req. 10(compression capability)	Supports native image compression (e.g., gzip, zip, tar)	Fully supports compression with advanced tools	Focused on security tools;
Req. 20(supports image processing)	Preconfigured for OpenCV and image libraries (Python, C++)	Supports image processing but requires more setup	Requires manual configuration
Req. 40(supports hashing utilities)	Includes sha256sum, md5sum, and related hash tools	Includes advanced hashing and encryption tools	Includes extensive hashing/security tools
Const. 30 (no user inputs/outputs during transfer)	supports headless operation	can run in server/headless mode	interactive security prompts possible
Std. 20 (IEEE standard for IT, file transfer, access, management)	Meets IEEE standard	Meets IEEE/ISO standard	Meets IEEE/ISO standard

When comparing Raspberry Pi OS, Ubuntu Linux, and Kali Linux, **Raspberry Pi OS** provides the most suitable balance for this system's design needs. It supports native compression formats such as gzip and zip, and comes preconfigured for **OpenCV** and other image processing libraries, ensuring smooth integration with the chosen detection algorithms. Additionally, it includes essential hashing utilities like **sha256sum** and **md5sum**, supporting data integrity checks without additional setup. Its ability to operate in **headless mode** aligns well with the requirement of no user interaction during data transfer. Overall, Raspberry Pi OS meets IEEE standards while offering optimized performance, simplicity, and compatibility specifically tailored for Raspberry Pi hardware.

SOFTWARE DESIGN TRADES 3

	Design Options		
Criteria (OS firmware with ESP 32 S3)	FreeRTOS	Arduino Framework	(NO OS)
<i>Req. 10 (multitasking)</i>	Can run several tasks like sending data and collaborating with Raspberry Pi without slowing down	Single loop only.	Single loop, line by line
Req. 20 (ESP-NOW Compatibility)	Full support for the latest ESP-NOW v2, larger packets, faster speed.	Supports older versions (v1) with smaller packets; slower for big files.	No built-in ESP-NOW; needs custom radio coding
Const. 30(No WIFI/Bluetooth)	Meets – uses ESP-NOW radio only, not networks.	Meets – same behavior as FreeRTOS	Meets – if coded correctly.
Const. 20 (Cost)	Free / open source (Espressif SDK).	Free / open source (Espressif SDK).	Free / open source (Espressif SDK).
Std. 20 (IEEE standard compliance)	Meets ISO/IEEE processing standards	Meets ISO/IEEE processing standards	No formal standards

When comparing FreeRTOS, the Arduino Framework, and a no-OS setup for the ESP32-S3, **FreeRTOS** offers the most robust and scalable solution. It supports true multitasking, allowing the microcontroller to handle simultaneous operations such as data transmission and coordination with the Raspberry Pi without performance loss. FreeRTOS also provides full compatibility with **ESP-NOW v2**, enabling faster data transfer and larger packet handling. While the Arduino Framework and no-OS options are simpler, they lack efficient task scheduling and rely on single-loop execution. Overall, FreeRTOS meets system constraints, supports ESP-NOW without Wi-Fi or Bluetooth, and complies with ISO/IEEE processing standards, making it the optimal firmware choice for reliable real-time operation as an optimized tradeoff.

SOFTWARE DESIGN TRADES 4

	Design Options		
Criteria (Encryption Method)	AES-128	AES-256	RSA
<i>Req. 10(Size in Bits)</i>	128	256	2048
Req. 90 (>10 kbps data rate)	Minimal processing overhead	Slightly higher overhead	Much slower due to key size
Req. 70 (transfer rate <600 s)	Fastest among tested algorithms	Moderate performance impact	Significant delay for large files
<i>Const. 20 (cost)</i>	Free, public standard	Free, public standard	
<i>Encryption Standards Std. 20 (IEEE standard for IT, file transfer, access, management)</i>	Meets NIST	Meets NIST	Meets PKCS

Among the evaluated encryption methods, **AES-128** offers the best trade-off between speed, security, and system efficiency. Its 128-bit key size provides strong encryption while maintaining minimal processing overhead, allowing data transfers to exceed 10 kbps with ease. Compared to AES-256, AES-128 delivers faster performance with lower computational demand, making it ideal for embedded systems like the ESP32-S3. RSA, though highly secure, introduces significant delays due to its large 2048-bit key size and is not suited for real-time data transfers. Overall, AES-128 meets NIST encryption standards, ensures data confidentiality, and achieves reliable performance within the project's time and cost constraints.

SOFTWARE DESIGN TRADES 5

	Design Options		
Criteria (Server Application/Mobile App Browser Access)	<i>Apache HTTP Server</i>	<i>Nginx</i>	<i>Node.js</i>
<i>Req. 10</i>	Supports static file hosting and HTTP/HTTPS	High-performance reverse proxy and file server	Supports dynamic web services and APIs
<i>Req. 90</i> (<i>> 10kbps data rate</i>)	Easily meets requirement (>1 Mbps typical)	Easily meets requirement (>1 Mbps typical)	Easily meets requirement (>1 Mbps typical)
<i>Req. 70</i> (<i>transfer rate < 600s</i>)	Efficient for medium-sized files	Optimized for large concurrent transfers	Optimized for asynchronous file transfer; fastest for real-time I/O
<i>Const. 10</i> (<i>prohibited use of WIFI and Bluetooth</i>)	Meets (local network only)	Meets (local network only)	Meets (local network only)
<i>Const. 20</i> (<i>cost</i>)	Free (open source)	Free (open source)	Free (open source)
<i>Std. 20</i> (<i>IEEE standard for IT, file transfer, access, management</i>)	Meets IEEE and IETF file transfer and HTTP standards	Meets IEEE and IETF standards	Meets IEEE and IETF web application standards

When comparing Apache HTTP Server, Nginx, and Node.js for server or browser-based access, **Node.js** stands out as the most suitable option for this design. It supports dynamic web services and APIs, enabling real-time data exchange between the ESP32-S3 and the Raspberry Pi. While Apache and Nginx excel at serving static files and handling large concurrent connections, Node.js provides superior asynchronous processing, making it the fastest for real-time input/output operations. All three meet data rate and security constraints while operating within a local network and at no additional cost. Overall, Node.js meets IEEE and IETF standards while offering flexibility, scalability, and high-speed performance ideal for interactive embedded applications.

DESIGN COMPONENTS

- The design components are measured and evaluated based on the technical reviews of the mission to ensure that each design choice meets the advised system functionality. Each component, both hardware and software, is selected to fulfill a specific role within the overall architecture including image processing, encryption, transmission and display. The hardware components, such as the Raspberry Pi 4 Model B and ESP32 S3 modules, are chosen for their processing capabilities and compatibility with ESP-NOW communication. Likewise, the software components, including OpenCV for image detection and FreeRTOS for firmware operation, were chosen to ensure a reliable system performance. Together, these components support the primary objective of securely transmitting and recovering the Death Star plans with adhering to constraints.

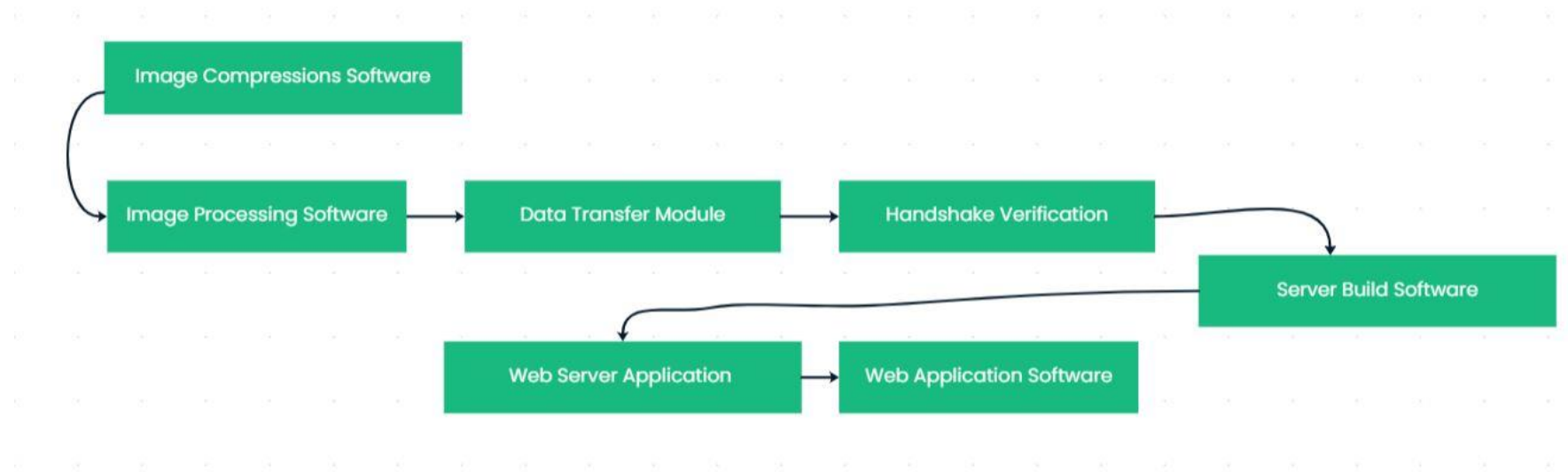
PROPOSED DESIGN

- Hardware
 - Storage Device(USB)
 - Raspberry Pi 4 Model B with SD Card
 - ESP32 S3 A
 - ESP32 S3 B
 - Laptop
 - Mobile Device



PROPOSED DESIGN

- Software
 - Image Compression Software
 - Raspberry Pi OS scripting
 - Image Processing Software
 - OpenCV/Python/Hough Circles
 - Data Transfer Module Software
 - FreeRTOS
 - Handshake Verification Software
 - AES-128/openssl
 - Server Build Software
 - Web Hosting Software
 - Web Application Software
 - Nginx
 - Authentication(Username and Password)Software
 - Nginx Configuration



HARDWARE INTERFACE

- To power the Raspberry Pi, a USB-C cable will be the primary power adapter for the Pi.
- Connecting the ESP32A(Transmitter) to the Raspberry Pi involves a physical connection by USB-A to USB-C cables to the ESP 32 A USB port. The cables will be supported by data cables for the Raspberry Pi OS recognition.
- As the transmitter and receiver are being communicated through ESP-NOW, The ESP32B will be plugged into the Laptop with a USB-C cable.

SOFTWARE INTERFACE

- Image Compression Software: Communicates with storage device provided with images to compress images running on Raspberry Pi OS using Python's Pillow Library.
- Image Detection Software: Communicates with image compression module using Open CV Hough circles library for output detection of red circles. It runs on Raspberry Pi OS using Python.
- MD5 Verification Software: Communicates with file transfer module to check md5sum with s
- Encryption Module: Communicates with both the File Transfer Module and Receiver Module using encrypted image data. Runs on both ESP32-A and ESP32-B using AES-128 in Python to ensure secure wireless transmission of each image chunk.
- File Transfer Software: Communicates with the Raspberry Pi OS over serial USB and with the Receiver Module wirelessly via ESP-NOW. Runs on ESP32-A using FreeRTOS in C++ to manage task scheduling for data transmission, encryption, and acknowledgment handling.
- Receiver Software: Communicates with the File Transfer Module using encrypted ESP-NOW packets and with the Web Server Application over USB serial. Runs ESP32-B using FreeRTOS in C++ to receive, decrypt, and forward image data to the laptop.
- Web Server Application: Communicates with the Receiver Module and Browser Interface using standard HTTP file transfers. Runs on the laptop using Node.js and Express.js to host a secure webpage for viewing uploaded images and performing user authentication.
- Browser Interface: Communicates with the Web Server Application using HTTPS requests and JSON responses. Runs on mobile and desktop browsers, allowing authenticated users to view detected images and complete two-factor authentication through a security question.

BUDGET/ COSTS

Description	Qty	Price Each	Total
USB 3.1 to USB C data cable	2	\$5.99	\$11.98
Esp-32 Microprocessor	2	\$15.00	\$30.00
Raspberry PI	1	\$30	\$30
USB2.0 to micro-USB	1	\$6.87	\$6.87
Total			\$90.83+taxes

All software tradeoffs are free open source and researched to be compatable with hardware.

RISKS AND MITIGATIONS

Risk 1: Shipping delay and component shortage
Mitigation: Order hardware parts early

		Unmitigated:					
L i k e l i h o o d:		1	2	3	4	5	
	5						
	4			X			
	3						
	2						
	1						
		Severity:					

		Mitigated:					
L i k e l i h o o d:		1	2	3	4	5	
	5	X					
	4						
	3						
	2						
	1						
		Severity:					

RISKS AND MITIGATIONS

Risk 2: Improper cable connections/ Short circuit
during hardware setup

Mitigation: Follow wiring diagrams carefully,
perform voltage testing, and test connections

	Unmitigated:					
L i k e l i h o o d:		1	2	3	4	5
	5					
	4					X
	3					
	2					
	1					
	Severity:					

	Mitigated:					
L i k e l i h o o d:		1	2	3	4	5
	5					
	4					
	3				X	
	2					
	1					
	Severity:					

RISKS AND MITIGATIONS

Risk 5: Software Bugs and Errors

Mitigation: Log Changes/ Implement Unit Testing Protocols

	Unmitigated:					
Likelihood:		1	2	3	4	5
	5	Green	Yellow	Red	Red	Red
	4	Green	Yellow	X Yellow	Red	Red
	3	Green	Yellow	Yellow	Yellow	Red
	2	Green	Green	Yellow	Yellow	Yellow
	1	Green	Green	Green	Green	Yellow
	Severity:					

	Mitigated:					
Likelihood:		1	2	3	4	5
	5	Green	X Yellow	Red	Red	Red
	4	Green	Yellow	Yellow	Red	Red
	3	Green	Yellow	Yellow	Yellow	Red
	2	Green	Green	Yellow	Yellow	Yellow
	1	Green	Green	Green	Green	Yellow
	Severity:					

TEST PLAN

Req No.	Obj No.	Test Method	Evaluation Method	Threshold	Objective
10	1,2	Upload 100 sample images and measure average output file size after compression.	Compare average compressed size to target value.	≤ 50 KB	≤ 40 KB
20	1	Run the circle-detection program on 100 test images (Open CV).	Count detected red circles vs expected number (10).	$\geq 90\%$ detection accuracy	100%
30	5	Login to web server on mobile device and attempt to load images.	Verify image display and access.	Successful display on ≥ 1 mobile browser	All major mobile browsers
40	2,3	Run md5sum on each image before and after transfer.	Compare hash values before and after transmission.	No mismatched hashes	0 failures across 100 images
50	3	Transmit files by connecting Raspberry Pi to ESP 32 S3 using ESP-NOW.	Count files successfully received.	$\geq 90\%$ success rate	100%
60	4	Load web page from public IP.	Visual confirmation of the correct display.	Accessible via local area network	Accessible via Internet
70	3	Run transfer script and count successes.	Verify all files present on the server.	≥ 10	≥ 100
80	5	Attempt to login with public access login versus private access login.	Check system response.	Blocks access when answer incorrect	No false positives
90	3	Measure transfer time for known file size.	Calculate data rate = file size / time.	≥ 10 kbps	≥ 20 kbps

DEVELOPMENT&TESTING TIMELINE

Milestone description	Category	Assigned to	Progress	Start	Days
Parts Ordered	On Track	All		11/24/2025	7
Raspberry pi has OS working	On Track	Coy Baker		1/24/2026	3
ESP-32 Setup	On Track	Trevor Kates		1/24/2026	20
Handshake of data transfer	On Track	Berrick Rudakemwa		2/4/2026	26
Data transfer is successful	On Track	All		2/28/2026	10
Image Processing	On Track	Trevor Kates		3/5/2026	14
Encription/AES	On Track	Brandie Ewing		3/2/2026	7
Webserver setup	On Track	Justin Leach		3/16/2026	7
Webserver Display	On Track	All		3/16/2026	8
Webserver Authentication	On Track	All		3/23/2026	9
Webserver Video Display	On Track	All		3/25/2026	10
Testing and Integration	On Track	All		3/26/2026	24
Final Revisions	Milestone	All			
Documentation and Presentation	On Track	All		4/20/2026	4

SCHEDULE

- **Nov 24 – Nov 28, 2025** – Parts Ordered
- **Dec 9, 2025 – Jan 11, 2026** – Semester Break (No hands-on work)
- **Jan 12 – Jan 17, 2026** – Raspberry Pi OS Installed and Configured
- **Jan 12 – Jan 22, 2026** – ESP-32 Setup and Initial Connection Testing
- **Jan 23 – Feb 8, 2026** – Handshake of Data Transfer Implemented
- **Feb 9 – Feb 15, 2026** – Data Transfer Successful Between Devices
- **Feb 16 – Mar 1, 2026** – Encryption / AES Implementation for Secure Communication
- **Mar 2 – Mar 8, 2026** – Image Processing (Detect and Analyze Death Star Images)
- **Mar 9 – Mar 15, 2026** – Webserver Setup on Rebel Server
- **Mar 16 – Mar 22, 2026** – Webserver Displays Images
- **Mar 23 – Mar 29, 2026** – Webserver Authentication (Obi-Wan Verification)
- **Mar 30 – Apr 5, 2026** – Webserver Video Display (Princess Leia Message)
- **Apr 6 – Apr 15, 2026** – Testing and Integration of All Components
- **Apr 16 – Apr 21, 2026** – Final Revisions and Bug Fixes
- **Apr 22 – Apr 25, 2026** – Documentation and Presentation Preparation

MILESTONES AND DELIVERABLES-

Milestones	Objectives
M1 – Parts Received	Physical Inventory List
M2 – ESP-Now Communication	Successful Packet Transfer Demo
M3 – Image Detection	Red Circle Detection Shown
M4 – File Transfer Success	10+ Images Transferred To Web App
M5 – Encryption/Decryption Verified	Output Matches Original File
M6 – Web Interface Operational	Live Web Demo On Laptop
M7 – Final Presentation	Full System Demonstration

QUESTIONS ?